

2025 Rocky Mountain Regional Solutions

The Judges

November 8, 2025

Credits

- Darcy Best
- Howard Cheng
- Ian DeHaan
- Ryan Farrell
- Zachary Friggstad
- Brandon Fuller
- Lawry Sorenson
- Nick Wu
- Charlie Zheng

Problem

- Given the initial and desired heading, give navigation directions.

Solution

One of many approaches:

- Convert string to degrees 0, 45, 90, 135, etc.
- Compute difference between desired and initial
- If negative, adjust and change direction
- If > 180 , adjust and change direction
- Special case of 0 and 180: straight and U-turn

Problem

- Simulate which person is holding which game.

Solution

- Keep a map/dictionary mapping each game to the person holding it.
- For each request, see who is holding it using the map and then update the map to indicate this person now holds the game.

Conveyor Belt Sushi

Problem

- Track sushi plates as they are placed onto and removed from a circular conveyor belt.

Solution

- The input is small enough for simulation to pass.
- Initialize empty arrays to represent the belt and each person's tab.
- Initialize time to 0.
- Loop over the input events.
- Advance time forward 1 second at a time to match the current time.
 - Check if the space in front of the chef is empty.
 - If so, place the next menu item onto the belt in front of the chef.
 - If the chef places the last menu item, set them back to the first.
 - Rotate the conveyor belt array clockwise.
- When someone takes a plate off of the belt, add it to their tab.

What Time Is It Mr. Fox?

Problem

- Simulate the game “What time is it Mr. Fox?”.

Solution

- It is too slow to move each sheep for each call.
- Instead, keep track of how many times each hour was called between “lunch time!” calls.
- Then when a “lunch time!” is called, the distance moved by sheep i can be computed in $O(h)$ time where $h = 11$ is the number of different possible calls apart from “lunch time!”.
- Running time: $O(n \cdot h \cdot c)$ where c is the number of “lunch time!” calls.

Problem

- A genie of rank K can grant K wishes.
- One of those wishes can be used to conjure another genie of rank $K' \leq K - C$.
- Can you get exactly W “real wishes”?

Solution 1

- Dynamic programming. Let $f(w, k)$ to mean whether we can make exactly W wishes if we have a genie of rank K .
- Base cases:

$$f(W, W) = \text{true}$$

$$f(W, K) = \text{false, if } W < K$$

- Recursion:

$$f(W, K) = \bigvee_{K'=1}^{K-1} f(W - (K - 1), K')$$

- Complexity: $O(WK^2)$

Solution 2

- If $W < K$, the answer is no.
- Greedy: use $K - 1$ real wishes and conjure a genie of rank $K - C$, until $K - C < 1$.
- If the total number of real wishes we can get from the greedy algorithm is at least W , we can make W wishes.
- Theorem: if $W > K$ and Aladdin can make W real wishes, he can also make $W - 1$ real wishes. Proof left as exercise.
- Complexity: $O(K)$

Hexadecimal Confusion

Problem

- Given hexadecimal numbers in which the pairs 0/D and 8/B can be confused, find the maximum/minimum possible sum.
- The original numbers have no leading 0s.

Solution

- To find maximum, replace all 0 by D and all 8 by B, add them.
- To find minimum, replace all B by 8, and all D by 0. . .

Hexadecimal Confusion

Problem

- Given hexadecimal numbers in which the pairs 0/D and 8/B can be confused, find the maximum/minimum possible sum.
- The original numbers have no leading 0s.

Solution

- To find maximum, replace all 0 by D and all 8 by B, add them.
- To find minimum, replace all B by 8, and all D by 0...
- But if that results in a leading 0, replace first digit by D...

Hexadecimal Confusion

Problem

- Given hexadecimal numbers in which the pairs 0/D and 8/B can be confused, find the maximum/minimum possible sum.
- The original numbers have no leading 0s.

Solution

- To find maximum, replace all 0 by D and all 8 by B, add them.
- To find minimum, replace all B by 8, and all D by 0. . .
- But if that results in a leading 0, replace first digit by D. . .
- Except the special case of a single digit of 0 or D: then 0 is allowed.

Spies vs. Spies

Problem

- Given a set of points, process queries asking each point to look N,S,E,W and eliminate all points it sees.

Solution

- For each row and column that has a point, maintain the points in that line in sorted order.
- When an instruction is given to look in a direction, it will see either a prefix or suffix of the points in the sorted order. Maintain pointers/indices into the sorted lists indicating the prefixes/suffixes that have been eliminated.
- Be careful, a point might have been eliminated from a vertical look so skip over them when processing a prefix/suffix of its row.
- Running time: $O(q + n \log n)$.

Poetic Tournament

Problem

- Find the top k out of n elements via interactive, pairwise comparisons.

Solution

- Fully sorting the n elements will need too many comparisons ($\mathcal{O}(n \log n)$).
- Intended solution is $\mathcal{O}(n \log k)$.
 - Iterate over the n elements.
 - Add each element to some data structure (sorted list, min-heap, or balanced binary tree).
 - Remove the smallest element from the data structure if its size is larger than k .
- Quickselect can also be used for an $\mathcal{O}(n)$ solution.

Problem

- Given a graph G with $k \leq 7$ nodes S such that each $v \in V - S$ is a neighbour of some node in S , determine if G has a 3-colouring.

Solution

- First, brute force all 3^7 possible ways to 3-colour S . For each $v \in V - S$, there are at most two possible colours for v since it has a neighbour in S .
- Use 2-SAT. For each v , associate `true` with one remaining colour and `false` with the other. Add clauses ensuring no edge receives the same colour at both endpoints.
- Running time: $O(3^k \cdot (n + m))$.

Energy Usage

Problem

- For each hour, given energy price p , energy required r , solar power generated s .
- Also, each hour there are a number of power buyback packages, which give c for selling d energy.
- There is also a battery with a given capacity, B .
- Find the minimum cost to operate the plant.

Solution

- Dynamic programming. Let $f(n, b)$ to mean the minimum cost to operate the factory to the end of the n -th hour and have b energy remaining in the battery. We also add a $(0,0)$ plan to every hour, and consider this the 0-th plan. This is equivalent to not choosing a plan.

Solution

- Base cases:

$$f(0, 0) = 0$$

$$f(0, b) = \infty \quad \text{if } b > 0$$

- Recursion:

Let $E_{\text{required}}(n, \Delta b, m)$ be the extra energy required to gain Δb battery level while utilizing the m -th plan at the n -th hour.

$$E_{\text{required}}(n, \Delta b, m) = r[n] - \Delta b - s[n] + d[n][m]$$

Solution

- Let $C(n, \Delta b, m)$ be the extra cost to gain Δb battery level while utilizing the m -th plan at the n -th hour.

$$C(n, \Delta b, m) = \begin{cases} -c[n][m], & \text{if } E_{\text{required}}(n, \Delta b, m) \leq 0, \\ -c[n][m] + E_{\text{required}}(n, \Delta b, m) \cdot p[n], & \text{if } E_{\text{required}}(n, \Delta b, m) > 0. \end{cases}$$

- Then we have

$$f(n, b) = \min_{b_i=0, \dots, B} \min_{m=0, \dots, M} C(n, b - b_i, m) + f(n-1, b_i)$$

- Complexity: $O(N \cdot M \cdot B^2)$

Draft Lottery

Problem

- Find the probability that various teams get one of the first k draft picks.

Solution

- Precompute all answers by generating all possible draft orderings.
 - Try all values of the first three teams, and compute the resulting order.
 - Note that ignoring a duplicate ball has the same effect as removing all of a team's balls when one is drawn, so the probability of the current order is $p = \frac{b_1}{s} \frac{b_2}{s-b_1} \frac{b_3}{s-b_1-b_2}$, where s is the total number of balls.
 - Add p to the probability of each team receiving their current draft pick.
- A prefix sum over the probabilities for each team yields the probability that they get any one of the first k picks.
- This approach has complexity $\mathcal{O}(n^4 + q)$.

Buying Jerseys

Problem

- Given a bipartite graph, find a color $c : \mathbb{N} \rightarrow \{0, 1, 2\}$ such that if $uv \in E$ either $c(u) \neq c(v)$ or $c(u) = c(v) = 2$. Among all such colorings, minimize $\sum_{v \in V} c(v)$.

Solution

- Find disjoint independent sets I_0, I_1 that minimize $|I_1| + 2 \cdot (n - |I_1| - |I_0|) = 2 \cdot n - (2 \cdot |I_0| + |I_1|)$. Equivalently, maximize $2 \cdot |I_0| + |I_1|$.
- Create a graph H with vertices $(v, 0)$ and $(v, 1)$ for each $v \in V$. Add edges $((v, 0), (v, 1))$ and for $uv \in E$, $((u, 0), (v, 0)), ((u, 1), (v, 1))$.
- Find maximum weighted independent set in H , weighting $(v, 0) = 2$ and $(v, 1) = 1$.
- H is bipartite, so this can be done with a standard flow formulation.
- Bounds set so that any reasonable flow implementation passes.

Team Practice

Problem

- Compute the number of ways students can switch teams during practice while meeting some constraints.

Solution

- The current state can be represented by the number of students of each type on team A (s_A, c_A).
- There are s_A ways to send one solver from A to B, $s - s_A$ ways to send a solver from B to A, and 1 way for no solvers to move between sessions. The same applies for coders as well.
- These transitions can be placed in a matrix M to advance the state X forward one session by computing MX . Note that $M^k X$ advances the state forward by k steps, and M^k can be found by repeated squaring.
- Jump the state forward to the sessions of the challenge problems, and mask any states where the problems are not solved.

Team Practice (cont.)

Solution

- However, with $M \in \mathbb{R}^{sc \times sc}$, this approach will TLE with time complexity $\mathcal{O}(s^3 c^3 p \log n)$.
- Note that the transitions for solvers and coders are linearly separable.
- The transition matrix, M , can then be separated into two transition matrices — M_s and M_c , while the state X can be represented as a matrix instead of a vector.
- The transition can then be established as $M_s^k X M_c^k$, where M_s performs row operations and M_c performs column operations on X .
- This approach has time complexity $\mathcal{O}((s^3 + c^3)p \log n)$.
- We also accept any solutions using Berlekamp–Massey to do sparse matrix exponentiation for $M^k X$ in $\mathcal{O}(s^2 c^2 p \log n)$.